Parcial 2 - Programación 3 - G2N

```
Universidad del Quindío
Programa de Ingeniería de Sistemas y Computación
Programación III - Parcial 2
Docente: Julián Esteban Gutiérrez Posada
```

Parcial 2

Programación funcional con Elixir | Recursividad, Streams y structs

Elijan 2 de los siguientes 4 problemas. Al menos uno debe ser el problema 1 o el 4 (Streams infinitos), y al menos uno debe ser el problema 1 o el 2 (structs). Por tanto, las combinaciones válidas son 1 y 2, 1 y 3, 1 y 4, o 2 y 4.

Requisitos generales de archivos y Streams

Estos requisitos aplican a los cuatro problemas, además de las condiciones particulares de cada uno.

1. Lectura en flujo. Lean todos los archivos de entrada con `File.stream!/1`, línea por línea; no usen `File.read!/1` ni carguen el texto completo para separarlo después.
2. Transformaciones perezosas. Limpien, filtren y conviertan las líneas mediante transformaciones perezosas, sin crear listas intermedias durante esa etapa. Una vez terminada, pueden construir mapas, listas o matrices si el algoritmo lo necesita.
3. Comentarios y espacios. Ignoren líneas en blanco y líneas cuyo primer carácter no blanco sea `#`. Conserven el número de línea física original para los reportes de errores.
4. Salidas por flujo. Creen o reemplacen cada archivo de salida en cada ejecución. Escriban líneas o fragmentos consecutivos a medida que se producen, sin formar previamente un único string con todo el resultado. Los SVG también se generan por fragmentos, con apertura, elementos y cierre.
5. Pantalla y archivo. Muestren en pantalla los resúmenes solicitados y guarden en los archivos todo el detalle especificado.
6. Errores de entrada. Si falta un archivo requerido, informen cuál falta y no terminen con una excepción sin manejar. Los registros inválidos se reportan con línea y motivo, cuando el problema lo indique.

7. Validación recursiva. La validación previa indicada para cada problema debe implementarse con funciones recursivas propias; dentro de esas funciones no se pueden utilizar `Enum` ni `Stream`.

8. Convenciones. Las coordenadas `{fila, columna}` comienzan en 0. En los archivos CSV de salida se usa `;` como separador. Los ejemplos son pequeños; prueben también casos vacíos, límites, errores y entradas de mayor tamaño.

---

1. Rutas de entrega en una red dirigida

Una empresa conecta centros de distribución por tramos de transporte dirigidos. Cada tramo tiene un tiempo en minutos y un costo en unidades monetarias. Desarrollen un programa que encuentre todos los itinerarios simples entre el centro inicial y el final de cada consulta.

El recorrido se construye recursivamente. Un centro no puede repetirse en un mismo itinerario y solo se pueden recorrer tramos en la dirección indicada. Para cada itinerario indiquen la secuencia de centros, los tramos utilizados, el tiempo total y el costo total. Para cada consulta informen el número de itinerarios y cuál o cuáles minimizan: (a) el número de tramos, (b) el tiempo y © el costo. Los tres criterios pueden producir soluciones diferentes; conserven todos los empates.

Archivos y Streams

Entrada `tramos.csv`. Cada línea tiene `origen,destino,minutos,costo`, con tiempos y costos enteros positivos.

```
# origen,destino,minutos,costo
A,B,8,5
B,D,7,4
A,C,6,2
C,D,12,1
B,C,3,2
C,B,4,1
A,A,2,1
A,B,9,7
```

Entrada `consultas.txt`. Una pareja `origen,destino` por línea.

```
A,D
B,A
```

Conviertan cada tramo bien formado en `%Tramo{origen, destino, minutos, costo}` mediante `Tramo.parse/1`, que devuelve `{:ok, tramo}` o `{:error, motivo}`. La búsqueda devuelve itinerarios representados con `%Itinerario{centros, minutos, costo}`. El grafo se arma después de consumir la transformación perezosa de líneas.

Salidas. `itinerarios.txt` contiene un bloque por consulta, cada itinerario numerado y los tres criterios óptimos. `errores_tramos.txt` registra líneas cuyo formato o datos sean incorrectos. Si un centro consultado no existe, se informa y no se inicia la búsqueda. En pantalla se muestra el total de itinerarios por consulta.

```
=== A !’ D ===
Itinerario 1: A !’ B !’ D | tramos: 2 | 15 min | costo: 9
Itinerario 2: A !’ B !’ C !’ D | tramos: 3 | 23 min | costo: 8
Itinerario 3: A !’ C !’ D | tramos: 2 | 18 min | costo: 3
Itinerario 4: A !’ C !’ B !’ D | tramos: 3 | 17 min | costo: 7
Total: 4 itinerarios
Menos tramos: 1, 3
Menor tiempo: 1
Menor costo: 3
```

Validación previa con recursión: depurar la red

Antes de construir el grafo, recorran recursivamente los tramos válidos y descarten los tramos que salen y llegan al mismo centro y las repeticiones de la misma pareja en la misma dirección; si cambian tiempo o costo, se conserva la primera aparición. Los tramos en direcciones contrarias son distintos y pueden coexistir. Registren cada descarte en `depuracion_tramos.txt` con línea física, contenido y motivo. La validación debe conservar el número de línea original asociado al struct.

Stream infinito: centros alcanzables

Para cada consulta, construyan un Stream infinito de conjuntos de centros alcanzables: el paso 0 contiene únicamente el origen; el siguiente añade todos los destinos de los tramos que salen de los centros ya alcanzados. Consuman solamente los pasos necesarios hasta detectar el primer estado repetido consecutivo. Escriban los estados distintos y el paso donde se estabiliza en `alcance_centros.txt`.

Si el destino no es alcanzable, indiquen «sin itinerarios» y omitan la búsqueda exhaustiva. Expliquen en la sustentación por qué la secuencia converge aunque el Stream sea infinito.

---

2. Identificación de regiones en un mapa de colores y exportación SVG

Un mapa rectangular está representado por una cuadrícula de símbolos. Dos casillas del mismo símbolo pertenecen a la misma región cuando se conectan por desplazamientos horizontales o verticales; las diagonales no unen regiones. El símbolo `.` es fondo y no forma regiones.

El programa debe encontrar recursivamente todas las regiones, sin recorrer dos veces una casilla ya asignada. Para cada región determine su símbolo, área (cantidad de casillas), perímetro (lados que tocan el borde o una casilla de otro símbolo) y caja envolvente `{fila_min, columna_min}` a `{fila_max, columna_max}`. Muestre el número de regiones y las de mayor área y mayor perímetro, incluyendo empates.

Archivos, structs y Streams

Entrada `mapa.txt`. Una fila por línea; los símbolos están separados por espacios.

```
A A . B B
A . . B .
. . C . .
C C C . B
C . . B B
```

Entrada `paleta.csv`. Formato `simbolo,nombre,color_hex`; cada símbolo aparece una sola vez. El símbolo `.` también se declara, para dibujar el fondo.

```
A,agua,#2563EB
B,bosque,#16A34A
C,cultivo,#F59E0B
.,fondo,#F1F5F9
```

Definan al menos `%Color{simbolo, nombre, hex}` y `%Region{simbolo, celdas, area, perimetro}`. Para construir las regiones utilicen una función recursiva que explore vecinos y otra que recorra las casillas aún no visitadas. Pueden almacenar el mapa en una estructura después de leerlo con Streams; no lean primero todo el texto como un solo string.

Salida `regiones.csv`. Una línea por región con `id;simbolo;area;perimetro;desde;hasta`. Asignen ID empezando en 1, según la primera casilla de la región encontrada al recorrer filas y columnas en orden ascendente.

```
id;simbolo;area;perimetro;desde;hasta
1;A;3;8;{0,0};{1,1}
2;B;3;8;{0,3};{1,4}
3;C;5;12;{2,0};{4,2}
4;B;3;8;{3,3};{4,4}
```

Salida `mapa_regiones.svg`. Construyan un SVG válido: cada casilla es un rectángulo de 40 × 40 píxeles con el color declarado en `paleta.csv`, borde visible y texto con el ID de región en al menos una de sus casillas. El SVG debe usar ancho `40 × columnas` y alto `40 × filas`, y mostrar el fondo. Generen el encabezado, los rectángulos y el cierre progresivamente, no concatenando el documento completo en memoria. Escriban también `resumen_regiones.txt` con número de regiones y máximos. Escapen correctamente cualquier texto que incorporen al XML.

Validación previa con recursión: mapa y paleta

Verifiquen recursivamente que todas las filas tengan igual longitud, no existan filas vacías, que cada símbolo del mapa esté definido en la paleta, que `.` exista en la paleta y que los colores usen la forma `#RRGGBB`. Detecten símbolos repetidos en la paleta. Reporten todos los problemas encontrados en `validacion_mapa.txt`, indicando fila, columna o línea, y no calculen regiones ni SVG mientras haya errores.

3. Expresiones infijas: conversión a postfija y evaluación recursiva

Desarrollen un intérprete de expresiones en notación infija, con paréntesis, que produzca la notación postfija y calcule su resultado. Debe manejar expresiones aritméticas, relacionales y lógicas, incluso combinadas, y respetar la precedencia y los tipos de datos.

Se aceptan enteros no negativos (`0`, `1`, `25`, …), los literales `true` y `false`, paréntesis y estos operadores:

Precedencia (de mayor a menor):

1. `!` — negación lógica unaria, asociativa a la derecha; produce booleano.
2. `*`, `/` — multiplicación y división; producen número.
3. `+`, `-` — suma y resta; producen número.
4. `<`, `<=`, `>`, `>=` — comparaciones de orden; producen booleano.
5. `==`, `!=` — igualdad y desigualdad; producen booleano.
6. `&&` — conjunción lógica; produce booleano.
7. `||` — disyunción lógica; produce booleano.

Los operadores de una misma prioridad son asociativos a la izquierda, salvo `!`. `+`, `-`, `*`, `/` y las comparaciones de orden requieren números; `!`, `&&` y `||` requieren booleanos; `==` y `!=` comparan dos valores del mismo tipo. La división produce número (posiblemente decimal); una división por cero es error. No se exige signo negativo unario, números decimales en la entrada, variables, ni evaluación con cortocircuito. Estas restricciones deben documentarse al usuario.

La solución debe usar recursión para (a) convertir los tokens infijos a postfijos utilizando pilas, sin `Code.eval_string/1` ni funciones equivalentes; y (b) evaluar los tokens postfijos recursivamente con una pila de valores tipados. Debe detectar y explicar expresiones inválidas, por ejemplo operandos ausentes, paréntesis mal ubicados, operadores incompatibles con tipos o división por cero.

Archivos y Streams

Entrada `expresiones.txt`. Una expresión por línea; se respetan las reglas generales para comentarios y líneas vacías.

```
(8 + 4) * 2
(7 * 2) >= (20 - 3)
!(3 > 5) && (2 + 2 == 4)
10 / (5 - 5)
```

Tokenicen respetando operadores de dos caracteres (`<=`, `>=`, `==`, `!=`, `&&`, `||`) y contando posiciones de caracteres, no desplazamientos de bytes. El texto se lee perezosamente con `File.stream!/1` y cada expresión se procesa individualmente sin guardar todas las expresiones antes de resolverlas.

Salida `resultado_expresiones.csv`. Formato `linea;infija;postfija;resultado;estado`; el estado es `OK` o `ERROR`. En una línea errónea, escriban el motivo en el campo resultado; si no se pudo generar una postfija válida, ese campo queda vacío.

```
linea;infija;postfija;resultado;estado
1;(8 + 4) * 2;8 4 + 2 *;24;OK
2;(7 * 2) >= (20 - 3);7 2 * 20 3 - >=;false;OK
3;!(3 > 5) && (2 + 2 == 4);3 5 > ! 2 2 + 4 == &&;true;OK
4;10 / (5 - 5);10 5 5 - /;división por cero;ERROR
```

También creen `errores_expresiones.txt`, con línea física, expresión original y motivo, y muestren en pantalla cuántas se evaluaron correctamente y cuántas tuvieron error.

Validación previa con recursión: estructura léxica

Antes de convertir cada expresión, implementen recursivamente una validación que detecte caracteres no permitidos, paréntesis desbalanceados y expresiones vacías. La validación debe detener la conversión de esa línea si hay errores y mostrar la posición del primero. Los errores sintácticos restantes y de tipos pueden descubrirse durante la conversión o evaluación; nunca deben causar una excepción sin manejar.

---

4. Coloración de grafos con generación perezosa

Una red tiene nodos unidos por aristas no dirigidas. Colorear la red con `k` colores consiste en asignar a cada nodo un entero de 1 a `k` sin que dos nodos vecinos reciban el mismo color. Para cada `k` solicitado, encuentren todas las coloraciones válidas por backtracking recursivo y cuéntenlas.

La búsqueda recursiva debe producir un Stream de soluciones: no se permite construir antes una lista con todas las coloraciones. Escriban cada solución cuando el Stream la entregue, sin conservar las anteriores. Procesen los nodos por orden de aparición en el archivo de entrada y prueben los colores en orden ascendente, para que la primera solución sea reproducible.

Archivos y Streams

Entrada `nodos.csv`. Formato `id,x,y`. Las coordenadas son enteros no negativos destinados al dibujo SVG.

```
A,60,60
B,180,60
C,180,180
D,60,180
```

Entrada `aristas.csv`. Formato `nodo_a,nodo_b`.

```
A,B
B,C
C,D
D,A
A,C
```

Entrada `colores.txt`. Un entero positivo `k` por línea.

```
2
3
4
```

Salidas `coloraciones_k<K>.txt`. Un archivo para cada `k`, con las asignaciones numeradas (`A=1;B=2;C=3;D=2`) y el total al final. Para el grafo del ejemplo, hay 0 coloraciones con 2 colores, 6 con 3 colores y 48 con 4 colores. Indiquen esos totales en pantalla.

Salida `primera_coloracion_k10.svg`. Obtengan solamente la primera solución con 10 colores, sin generar las demás, y dibujen la red en SVG: cada arista es una línea; cada nodo es un círculo situado en `(x,y)`, con su ID y un relleno visual que distinga los colores. Pueden definir una paleta de colores de dibujo en el código; la asignación lógica sigue siendo el entero 1…10. Usen un lienzo suficientemente grande y escriban los elementos del SVG progresivamente.

Validación previa con recursión: red y propuestas

Antes de buscar, verifiquen recursivamente que no haya IDs de nodo duplicados, aristas con nodos desconocidos, bucles `A,A` ni aristas repetidas (incluidas las invertidas). Registren todos los problemas en `validacion_red.txt` y no busquen coloraciones con una red inválida.

Entrada `propuestas.txt`. Cada línea indica `k;id=color;id=color;...`. Por ejemplo:

```
3;A=1;B=2;C=3;D=2
3;A=1;B=2;C=1;D=2
```

Verifiquen cada propuesta recursivamente: deben aparecer exactamente los nodos de la red, cada color debe estar en `1..k` y ninguna arista debe conectar nodos del mismo color. Escriban en `verificacion_colores.txt` si es válida y, en caso contrario, todos los conflictos y faltantes encontrados.

Stream infinito: número mínimo de colores

Construyan el Stream infinito `k = 1, 2, 3, …` y transfórmenlo perezosamente en `{k, total_de_coloraciones}`. Consúmanlo hasta encontrar el primer `k` con total mayor que cero, incluyéndolo en `secuencia_colores.csv`; no calculen el siguiente. En el grafo de ejemplo, el primer valor factible es `k = 3`. Expliquen por qué el Stream es infinito pero el consumo es finito si existe una solución.

Entrega y sustentación

Cada grupo debe entregar:

1. Un repositorio de GitHub público o compartido con el docente, con el código Elixir, los archivos de entrada utilizados y las instrucciones necesarias para ejecutar los dos problemas elegidos. No se reciben archivos sueltos.
2. Un informe PDF que incluya la URL del repositorio y las secciones que se especifican enseguida.

Datos, resultados y pruebas

Para cada problema y cada apartado (búsqueda o procesamiento principal, validación recursiva, generación de SVG y Streams infinitos, cuando correspondan), muestren archivos de entrada, archivos de salida y pruebas realizadas. Incluyan al menos un caso válido, uno inválido y uno límite. El SVG entregado debe abrir correctamente en un navegador cuando el problema lo requiera.

Reflexión sobre el uso de inteligencia artificial

Si usaron IA, respondan de forma concreta; si no la usaron, expliquen cómo resolvieron las dudas.

1. Herramientas usadas. ¿Qué asistentes, agentes o extensiones emplearon y en qué fases del parcial?
2. Interacción decisiva. Documenten una consulta a la IA, su respuesta y cómo verificaron o modificaron esa propuesta.
3. Enfoques nuevos. Expliquen una sugerencia distinta de lo trabajado en clase; qué aporta, si se usó y qué aprendieron.
4. Balance de aprendizaje. ¿Qué partes comprenden sin ayuda, cuáles siguen siendo difíciles y qué harían diferente en el próximo trabajo?

Video de sustentación individual en donde explique funciones no triviales y diferentes a la de sus compañeros.